

# Reviewer Pack — Minimal Rank of Tropicalized Divisors over Planar Curves vs ...

Entry 8b94d4fadfb5 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-24 07:49:49 UTC

## Minimal Rank of Tropicalized Divisors over Planar Curves vs BP\_ReadTwice Tensor Width

Entry ID: 8b94d4fadfb5 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-24 07:49:49 UTC

### 1. Conjecture

**Field A** (mathematical branch): Tropical Geometry **Field B** (complexity object): Communication Complexity (BP\_ReadTwice)

#### Statement:

[“For a given planar curve  $C$  with  $n$  algebraic points, the minimal rank of its tropicalized divisor is upper-bounded by a function that grows at most linearly with the tensor width  $\rho(P)$  of any read-twice BP  $P$  computing the sign of the curve’s intersection number.”, ‘Equivalently, for all planar curves  $C$  and corresponding BPs  $P$ , there exists a constant  $c$  such that  $\text{MinRank}(\text{TropicalDivisor}(C)) \leq c * \rho(P)$ .’, ‘Finally, if there exists a curve with  $n$  algebraic points and a BP  $P$  of size  $s$  such that  $\text{MinRank}(\text{TropicalDivisor}(C)) > c * \rho(P)$ , then the communication complexity of deciding whether the intersection number is zero for  $P$  is at least  $\Omega(s \log s)$ .’]

#### Rationale (proposer’s reasoning):

[“Tropical geometry provides a bridge between algebraic curves and computational structures, which may reveal hidden relationships between geometric properties and communication complexity. The minimal rank of tropicalized divisors captures the geometric information encoded in planar curves, while BP\_ReadTwice tensor width measures the complexity of their computation.’, ‘This conjecture proposes that the minimal rank of tropicalized divisors can be used to bound the complexity of BP\_ReadTwice computations, potentially leading to new algorithms or lower bounds in communication complexity.’]

**Taxonomy category:** BP\_READTWICE (status at proposal time: )

### 2. Pre-registration (Popper-style)

**Hash (SHA-256 prefix):** ab8be782be0379e3

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

**Acceptance criterion:**

The minimal rank of tropicalized divisors over planar curves is upper-bounded by a function linearly related to the tensor width of BP\_readTwice, with a correlation coefficient greater than zero indicating support and less than or equal to -0.5 indicating falsification.

### 3. Barrier filter (F1)

**Final:** PASS

| Barrier        | Verdict | Confidence | LLM <sub>1</sub> | LLM <sub>2</sub> |
|----------------|---------|------------|------------------|------------------|
| RELATIVIZATION | SAFE    | 0.50       | UNCERTAIN        | UNCERTAIN        |
| NATURAL PROOFS | SAFE    | 0.50       | UNCERTAIN        | UNCERTAIN        |
| ALGEBRIZATION  | SAFE    | 0.50       | UNCERTAIN        | UNCERTAIN        |
| KARP_LIPTON    | SAFE    | 0.50       | UNCERTAIN        | UNCERTAIN        |

### 4. Novelty audit

**Verdict:** NOVEL against 3 hits across arXiv + Semantic Scholar + ECCC

**Search queries (3):** - "tropical geometry" AND "BP\_ReadTwice tensor width" AND minimal rank" - "tropicalized divisor" AND planar curve" AND communication complexity BP\_ReadTwice" - "intersection number sign" AND constant bound" AND tropical geometry BP\_ReadTwice"

**Top relevant hits considered:** - [http://arxiv.org/abs/1802.02440v2] Tropicalized quartics and canonical embeddings for tropical curves of genus 3 - [http://arxiv.org/abs/2210.09696v2] Tropical lifting problem for the intersection of plane curves - [http://arxiv.org/abs/1701.06579v2] Brill-Noether theory for curves of a fixed gonality

### 5. Empirical test harness

**Multi-seed protocol:** 5 seeds (11, 23, 37, 53, 71)

**Sandbox:** isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

#### 5.1 Generated Python source

```
import random
import math

def gaussian_elimination(matrix):
    rows, cols = len(matrix), len(matrix[0])
    for i in range(rows):
        max_row = i + max(range(i, rows), key=lambda j: abs(matrix[j][i]))
        matrix[i], matrix[max_row] = matrix[max_row], matrix[i]
        for j in range(rows):
            if i != j:
                factor = matrix[j][i] / matrix[i][i]
                for k in range(cols):
                    matrix[j][k] -= factor * matrix[i][k]
    return matrix

def determinant(matrix):
    rows, cols = len(matrix), len(matrix[0])
    det = 1
    for i in range(rows):
        if matrix[i][i] == 0:
            return 0
```

```

        for j in range(i + 1, rows):
            factor = matrix[j][i] / matrix[i][i]
            for k in range(cols):
                matrix[j][k] -= factor * matrix[i][k]
            det *= matrix[i][i]
    return det

def tensor_width(bp_size):
    # Placeholder function to simulate BP_ReadTwice tensor width calculation
    return bp_size

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.choice([10, 15, 20, 25, 30])
    instances_tested = 0
    total_rank = 0
    total_width = 0

    for _ in range(6): # Ensure at least 30 instances per seed
        points = [(random.uniform(-1, 1), random.uniform(-1, 1)) for _ in range(n)]
        tropical_divisor = [sum([abs(x[0] - p[0]) + abs(x[1] - p[1]) for p in points]) for x in points]
        min_rank = len(tropical_divisor) - max(tropical_divisor)

        bp_size = n * (n - 1) // 2 # Placeholder BP size
        width = tensor_width(bp_size)

        total_rank += min_rank
        total_width += width
        instances_tested += 1

    avg_rank = total_rank / instances_tested
    avg_width = total_width / instances_tested
    correlation_coefficient = (instances_tested * sum(avg_rank * avg_width for _ in range(instances_tested)) -
                              sum(avg_rank) * sum(avg_width)) / math.sqrt((instances_tested * sum(avg_rank ** 2) -
                                                                              (sum(avg_rank) ** 2) *
                                                                              (instances_tested * sum(avg_width ** 2) -
                                                                              (sum(avg_width) ** 2) *
                                                                              instances_tested)))

    conjecture_holds = correlation_coefficient > 0
    counterexample = "" if conjecture_holds else "correlation_coefficient"

    return {
        "metric_name": "Correlation Coefficient",
        "metric_value": correlation_coefficient,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2**i - 1 for i in range(5, 8)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, ...run_trial output...}}")
        results.append(result)

```

```

mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value)**2 for r in results) / len(results))
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
elif any(not r["conjecture_holds"] for r in results):
    first_failing_seed = next(s for s, r in zip(seeds, results) if not r["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample={r['counterexample']} first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE mapping_undefined")

```

## 6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

## 7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_970c1b6a.py", line 89, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_970c1b6a.py", line 69, in run_trial
    sum(avg_rank) * sum(avg_width)) / math.sqrt((instances_tested * sum(avg_rank**2) -
sum(avg_rank)**2) *
    ^^^^^^^^^^^^^
TypeError: 'float' object is not iterable

```

## 8. Critic adversarial review

**Critic verdict:** CHALLENGE

**Critic reasoning:**

skipped: test crashed before producing data

## 9. Final verdict & safety rail

**Verdict:** INCONCLUSIVE

**Reasoning:**

The test code crashed before producing data, which prevents us from evaluating the conjecture's validity based on the pre-registered support conditions. | next: Investigate and fix the error in the test code to allow for a proper evaluation of the conjecture.

## 11. Audit log (LLM calls)

**Total LLM calls:** 9

| # | Phase           | Provider      | Model       | Tokens in | out | Latency (ms) |
|---|-----------------|---------------|-------------|-----------|-----|--------------|
| 1 | propose         | ollama_remote | glm4:latest | 0         | 0   | 12123        |
| 2 | preregistration | ollama_remote | glm4:latest | 0         | 0   | 5528         |
| 3 | novelty         | ollama_remote | glm4:latest | 0         | 0   | 4802         |

| # | Phase    | Provider      | Model            | Tokens in | out | Latency (ms) |
|---|----------|---------------|------------------|-----------|-----|--------------|
| 4 | novelty  | ollama_remote | glm4:latest      | 0         | 0   | 5543         |
| 5 | test_gen | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 26750        |
| 6 | test_gen | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 7179         |
| 7 | test_gen | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 12544        |
| 8 | test_gen | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 11471        |
| 9 | judge    | ollama_remote | glm4:latest      | 0         | 0   | 8435         |

**Totals:** 0 input tokens, 0 output tokens, ~\$0.0000 cost, 94375 ms total latency. Provider mix: {'ollama\_remote': 9}

(full prompt+response transcripts available in `research/audit/8b94d4fadfb5.jsonl`)

## 12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/8b94d4fadfb5.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

**Sandbox file:** `research/pvsnp_sandbox/test_*.py` (per-cycle)

**Replay tarball:** `research/replay/8b94d4fadfb5.tar.gz` (if generated)